

Skill Builder Feedback Report

Position: Skill Assessment - AI Engineer

Date: 5/24/2026

Session ID: dlai_interview_452d3d7110594515ab9c48f9131e85cc



Skill Builder Overview

SUMMARY

We discussed your multi-project portfolio, focusing on a Text-to-SQL system for schema mapping and a RAG pipeline built with Azure AI Search and LangGraph. You demonstrated a strong focus on production reliability through the use of golden datasets and granular debugging tools.

STANDOUT STRENGTHS

Your implementation of a golden dataset paired with Arize Phoenix for node-level debugging shows a sophisticated approach to evaluation. Additionally, your use of practical safety checks for SQL generation, such as the 'WHERE 1=0' syntax validation, highlights a focus on system stability.

KEY DEVELOPMENT AREAS

The primary growth opportunities lie in automating schema pruning for Text-to-SQL to move beyond manual topic buckets and implementing advanced retrieval techniques like semantic ranking to bridge the terminology gap in RAG.

RECOMMENDED FOCUS

Transition your Text-to-SQL schema pruning from manual topic-to-table mapping to a dynamic retrieval mechanism, such as a Knowledge Graph or semantic search over metadata, to handle scalability beyond 100 tables.



Recommended Learning Path

Curated based on your skill gap analysis



Evaluating AI Agents Short Course

This course directly aligns with your use of Arize Phoenix and will help you formalize your evaluation experiments and observability practices.



Knowledge Graphs for AI Agent API Discovery

Short Course

Since you mentioned moving toward a graph database for schema retrieval, this course provides the technical foundation for using graphs to help agents discover the right data structures.



AI Agents in LangGraph Short Course

This will help you deepen your expertise in the framework you are already using, specifically around managing state and integrating agentic search.





Skill Assessments

RAG Evaluation & Observability

Key Evidence:

- You described using a golden dataset to verify that the system returns the desired results.
- You mentioned using Arize Phoenix to debug the pipeline and identify issues at the specific module, agent, or node level.
- You identified specific failure modes such as 'context missing', 'incomplete context', and 'buggy generation' during iterative testing.

Path to Next Level:

To advance your evaluation framework, integrate the RAGAS library to calculate automated metrics like faithfulness and answer relevancy, which can supplement your golden dataset. Since you are using Azure AI Search, enable the Semantic Ranker to perform L2 re-ranking on your vector results, which often resolves the 'top-K' precision issues you noted. For deeper observability, implement OpenInference tracing within your LangGraph nodes to visualize the exact flow of data and latency across your separate reasoning and retrieval graphs.

Text-to-SQL Architecture

Key Evidence:

- You explained a schema linking strategy using metadata blobs containing table, column, and formula information.
- You described a safety mechanism using a 'WHERE 1=0' clause to validate SQL syntax without executing full queries.
- You identified a scalability wall where passing 100+ tables in a single context window becomes unfeasible.

Path to Next Level:

Move beyond manual topic buckets by implementing a 'Schema Retriever' that uses semantic search to find relevant table and column descriptions based on the user's natural language intent. Explore the use of a Knowledge Graph (using Neo4j or similar) to map relationships between tables, allowing the LLM to discover join paths dynamically rather than relying on a flat metadata blob. Finally, implement a multi-stage prompt that first selects the schema and then generates the SQL to reduce the noise in the final generation step.

Agentic Orchestration (LangGraph)

Key Evidence:

- You confirmed using LangGraph to manage different agents and nodes within your application.
- You described an architectural choice to keep reasoning and retrieval steps in separate graphs for modularity.
- You mentioned using meta-prompting and query expansion as part of your agentic workflow to improve search accuracy.

Path to Next Level:

To enhance your multi-graph architecture, implement a 'Supervisor' node that can route tasks between your reasoning and retrieval graphs based on the complexity of the user query. Explore persistent state management in LangGraph to allow for multi-turn conversations where the agent remembers previous schema selections or retrieval filters. You could also integrate a 'Human-in-the-loop' node specifically for the SQL generation phase, allowing an admin to approve or edit complex queries before they hit the production database.